TEC | Tecnológico de Costa Rica

Informe técnico

Asignación de Horarios Universitarios
con GNU Prolog

Lenguajes de Programación

Estudiante

Daniel Araya Pérez
Carné 2025078995

Profesor

Eddy Ramírez Jiménez

1. Modelo y representación

El programa asigna cada sesión de un grupo a un aula, un día y una franja inicial. Una sesión de duración d_c ocupa d_c franjas seguidas y debe terminar dentro del día. Para una duración positiva, sus posibles ubicaciones son

$$X_{c,i} \in A \times D \times \{1, \ldots, |F| - d_c + 1\},$$

donde A son las aulas, D los días, F las franjas diarias y C los pares curso–grupo. Si el grupo c tiene s_c sesiones semanales, el número total de sesiones y el espacio de búsqueda sin poda son

$$n = \sum_{c \in C} s_c, \qquad \Phi = \prod_{c \in C} \left(|A|\,|D|\,(|F| - d_c + 1)\right)^{s_c}.$$

El programa calcula Φ como una aproximación en punto flotante. Esta cantidad no es el número de nodos que visita la búsqueda.

La instancia se guarda con los siguientes hechos de GNU Prolog.

```
instanceSetting(Key, Value)
room(Room, Capacity, Type)
teacher(Teacher, Name, MaxSlots, Availability)
group(Course, Group, Name, Enrolled, WeeklySessions, Duration, Requirement)
teaches(Course, Group, Teacher)
```

Los identificadores se guardan como listas de bytes UTF-8. Cada sesión tiene un número dentro de su grupo. Las dos estrategias entregan el mismo formato de horario.

```
session(Course, Group, SessionOrdinal)
slot(Room, Day, Start)
assignment(Course, Group, Teachers, Room, Day, Start, Duration)
```

`Teachers` contiene todos los profesores del grupo, que participan en la misma sesión. Una relación de docencia repetida no duplica profesores ni sesiones.

2. Restricciones

2.1. Restricciones duras

Todo horario debe cumplir las siguientes reglas.

- Dos sesiones no pueden compartir una franja del mismo día si pertenecen al mismo grupo, tienen un profesor en común o usan la misma aula.
- El aula debe tener capacidad suficiente y un tipo compatible. Una sesión de teoría puede usar aulas de teoría o mixtas. Una de laboratorio puede usar aulas de laboratorio o mixtas. Una de tipo mixto puede usar cualquiera de los tipos declarados.
- La sesión debe comenzar cuando todos sus profesores estén disponibles. Se revisa la franja de inicio, como indica el enunciado.
- La carga semanal de cada profesor no puede superar su máximo de franjas. Se calcula sumando sesiones semanales por duración de cada grupo que imparte.
- `aula_fija` obliga a usar un aula concreta. `prohibida` impide ocupar la franja indicada. `excluyentes` impide que dos grupos compartan una franja del mismo día.

La carga del profesor se conoce antes de buscar, porque el programa no elige qué grupos imparte.

2.2. Restricciones blandas y optimización

`prefiere` agrega su peso al costo si el grupo no tiene una sesión que empiece en el día y la franja indicados. `compacta` agrega su peso por cada hueco del grupo. Un hueco es una franja libre entre la primera y la última franja ocupada en un mismo día. Se cuentan todas las franjas que cubre cada sesión. Los huecos de profesores se muestran en la salida, pero no se suman al costo.

Con `--optimizar`, ambas estrategias usan *branch and bound*. Guardan el mejor horario completo encontrado y calculan una cota inferior con el costo de los grupos que ya tienen todas sus sesiones asignadas. Si esa suma es mayor o igual al mejor costo, dejan de explorar la rama.

La cota es válida para pesos no negativos que caben en los enteros de GNU Prolog. El costo de un grupo completo ya no cambia y los grupos pendientes solo pueden agregar un costo no negativo. En dominios finitos también se cuentan los grupos que la propagación deja completamente asignados. El resultado se marca como óptimo solo si la búsqueda termina sin alcanzar el límite.

3. Estrategias de búsqueda

3.1. Backtracking (BT)

Se guarda una lista de sesiones pendientes y otra de asignaciones. Para cada sesión se prueba una ubicación de su dominio. Si una opción falla, Prolog vuelve a una opción anterior.

La variante básica usa generación y prueba. Primero asigna todas las sesiones y luego verifica el horario. Conserva el filtro inicial por capacidad, tipo y disponibilidad, así como las reducciones de simetría. Esta variante se usa como referencia en las mediciones.

La variante con verificación anticipada revisa los conflictos con las sesiones ya asignadas antes de seguir. Así evita completar horarios que ya incumplen una regla. Las opciones compatibles se vuelven a calcular según la rama actual. Esta es la variante que utiliza normalmente el programa.

Los nodos, retrocesos y el mejor horario se guardan con `assertz/1` y `retract/1`, para conservarlos cuando Prolog retrocede.

3.2. Dominios finitos (FD)

Cada sesión usa tres variables enteras para el índice de aula, el índice de día y la franja inicial. Los índices de aula y día comienzan en 1 y siguen el orden de la instancia. El modelo se guarda de la siguiente forma.

```
fdSession(Session, RoomIndex, DayIndex, Start, Tuples)
indexMaps(Rooms, Days)
```

`fd_domain/3` fija los rangos y `fd_relation/2` permite solo las combinaciones de aula, día y franja que cumplen capacidad, tipo, disponibilidad, aula fija y franjas prohibidas.

Para evitar conflictos, se comprueba que las sesiones tengan distinto día, intervalos separados o, cuando basta, distinta aula. Estas comparaciones reducen los valores posibles durante la búsqueda. Los conflictos y el orden de simetría se manejan con comparaciones, sin listar todas las combinaciones permitidas. La carga de profesores se revisa antes de buscar porque es fija. Los costos blandos se calculan con predicados de Prolog.

Para saber qué ubicaciones siguen disponibles, FD prueba sus tres valores dentro de `findall/3`. El retroceso deshace cada prueba. `fd_labeling/2` asigna las variables que aún faltan, comenzando por el menor valor.

4. Heurísticas y poda

4.1. Orden de búsqueda

Las dos estrategias usan los siguientes criterios.

- **Orden original**. Se toman las sesiones en el orden de lectura.

- **MRV**. Se elige primero la sesión con menos ubicaciones posibles en la rama actual.
- **Grado**. Se elige primero la sesión con más restricciones con otras sesiones pendientes. Se cuentan aula, grupo, cada profesor compartido y exclusión.
- **Demanda**. Se elige primero el grupo con más inscritos y, en caso de empate, mayor duración.
- **LCV**. Se toma la primera sesión pendiente y se prueba primero la ubicación que deja más opciones a las demás.

Los empates conservan el orden original. MRV cuenta las combinaciones completas de aula, día y franja que aún son posibles. En FD, `ff` usa este mismo criterio. Se aplica a las ubicaciones completas y no a una sola variable de `fd_labeling/2`. Así, BT y FD comparan la misma clase de decisión.

4.2. Reducción de simetría

Se aplican dos reglas para evitar horarios que solo cambian el orden de sesiones o los nombres de aulas equivalentes.

- Las sesiones de un mismo grupo se ordenan por día y franja inicial, según su número semanal. BT revisa el orden al asignar. FD lo impone entre sesiones consecutivas. Para duración cero se permite el mismo inicio.
- Las aulas de igual capacidad y tipo se usan por primera vez en un orden fijo. Se excluyen las aulas que aparecen en `aula_fija`. La comparación usa la lista original de sesiones, aunque la heurística elija otro orden.

En FD, el orden de sesiones usa dos variables que pueden valer 0 o 1. Una indica un día anterior y la otra indica el mismo día con inicio anterior. `fd_all_different/1` obliga a que sean distintas. Como ambos casos no pueden ocurrir juntos, esto exige que uno se cumpla. El intercambio de aulas equivalentes conserva las restricciones y el costo.

5. Archivos, verificación y errores

`horarios.sh` invoca GNU Prolog. El programa lee el archivo en modo binario y procesa sus líneas con listas y recursión. Reconoce las secciones, elimina los espacios al inicio y al final de los campos y conserva los números de línea. No usa utilidades externas para leer la instancia.

La lectura se detiene ante el primer error de sintaxis. La validación revisa referencias, duplicados y rangos, y reúne los errores de coherencia. Como recibe todas las secciones, su orden no afecta las referencias. Los hechos de la instancia se reemplazan solo después de validar los datos.

Antes de buscar, el programa detecta grupos sin aula o ubicación permitida, sesiones que no caben y profesores con carga imposible. Que estos controles pasen no garantiza que exista una solución. Antes de escribir un horario, `checkSchedule/1` revisa de forma independiente las restricciones duras y la cantidad exacta de sesiones por grupo.

Si una instancia no tiene solución, el diagnóstico busca primero sin restricciones duras declaradas. Luego las agrega en el orden de lectura e identifica la primera que deja la instancia sin solución. Si falla desde el inicio, la causa está en las restricciones implícitas. No se busca un conjunto mínimo de restricciones.

La salida ordena las asignaciones y muestra horarios por grupo, profesor y, si se solicita, aula. La exportación a Prolog contiene un hecho `asignacion/7` por sesión, con los mismos datos del horario. Al terminar de escribir, se cierran los archivos.

Los errores de sintaxis y coherencia producen un reporte de instancia inválida. Los errores de archivos y las inconsistencias detectadas antes de buscar se informan en la terminal. Si se alcanza el límite y ya se encontró un horario completo, se conserva el mejor. No se entrega un horario parcial.

6. Resultados experimentales

6.1. Mediciones

Se usó GNU Prolog 1.4.5 en Ubuntu, con 128 MiB de pila global. Las búsquedas tuvieron un límite de 5000 nodos y cada proceso pudo ejecutarse hasta cinco segundos.

Cada prueba se repitió cinco veces en las comparaciones de heurísticas, ejecución compilada y optimización. En los tamaños crecientes y el caso FD sin restricciones blandas se repitió tres veces. Las tablas muestran la mediana, que es el valor central de las repeticiones.

`statistics/2` mide CPU y tiempo transcurrido durante la lectura, validación, preparación y búsqueda. La orden `time` mide el proceso completo, incluyendo el arranque del programa y la escritura de la salida. Las versiones compiladas se prepararon antes de medir. Cuando se ejecuta el diagnóstico, sus tiempos y contadores también se suman.

Un nodo es el estado inicial o una ubicación admitida antes de revisar orden, simetría, cota y continuación. Las ubicaciones descartadas por el filtro no cuentan. Un retroceso se cuenta cuando una opción termina sin interrumpir la búsqueda por solución o límite. Las pruebas temporales de las heurísticas consumen tiempo, pero no se cuentan como nodos.

Se usan N para nodos, B para retrocesos y T para CPU en segundos. Las métricas pedidas son

$$R = \frac{N}{T}, \qquad S = \frac{T_{\text{básico}}}{T}, \qquad E = \frac{N_{\text{básico}} - N}{N_{\text{básico}}}.$$

R es la tasa de exploración, S la aceleración y E la eficiencia de poda. En la tabla, E se muestra como porcentaje. No se calculan tasa ni aceleración cuando el tiempo medido es cero, ni se compara una búsqueda interrumpida con otra completa como si ambas hubieran resuelto la instancia.

6.2. Comparación de heurísticas

Se usó `instances/valid/constraints.dat`, con cinco sesiones y ocho franjas ocupadas, buscando la primera solución sin optimizar. En la tabla 1, BT usa verificación anticipada, salvo la variante básica.

Configuración	T (s)	N	B	R (s^{-1})	S	E (%)	Costo
BT-básico	0.081	1458	1452	18000	1.00	0.00	2
BT-original	0.002	6	0	3000	40.50	99.59	2
BT-MRV	0.005	6	0	1200	16.20	99.59	2
BT-grado	0.003	6	0	2000	27.00	99.59	2
BT-demanda	0.002	6	0	3000	40.50	99.59	2
BT-LCV	0.017	14	8	823.5	4.76	99.04	3
FD-original	0.002	5	0	2500	40.50	99.66	2
FD-MRV	0.003	5	0	1666.7	27.00	99.66	2
FD-grado	0.002	5	0	2500	40.50	99.66	2
FD-demanda	0.002	5	0	2500	40.50	99.66	2
FD-LCV	0.004	5	0	1250	20.25	99.66	2

Tabla 1. Resultados por estrategia y heurística

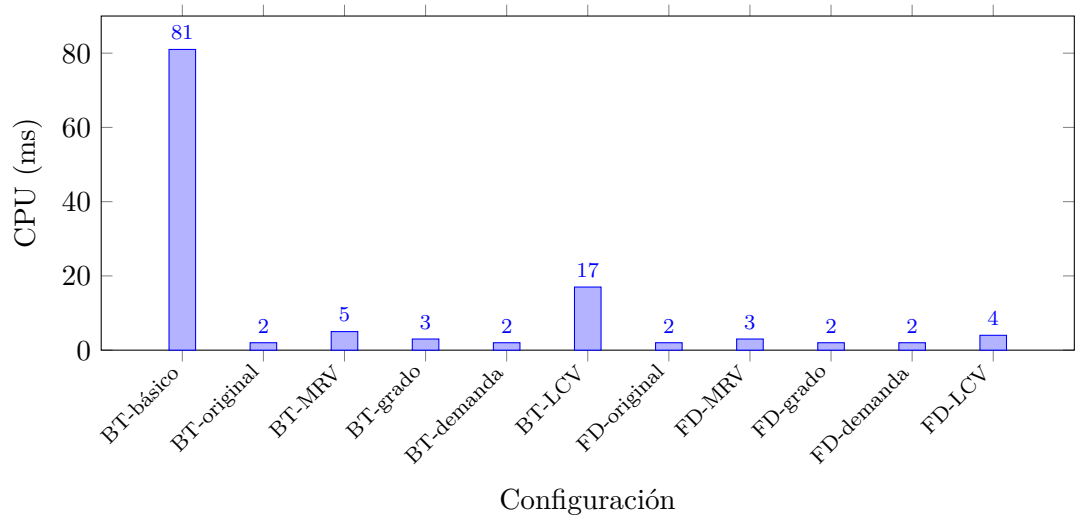


Figura 1. Tiempo mediano de CPU sobre la instancia común

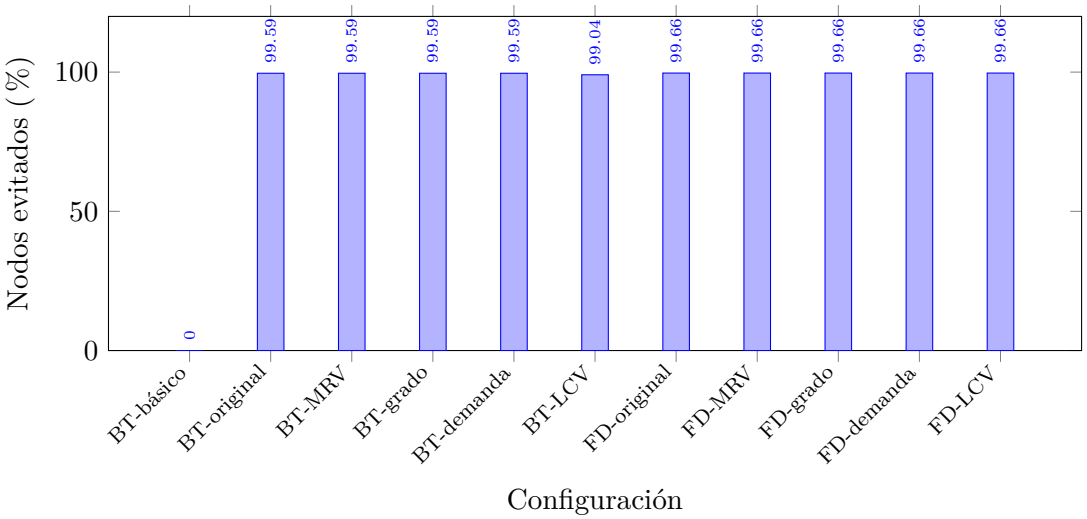


Figura 2. Porcentaje de nodos evitados

6.3. Ejecución interpretada y compilada

La tabla 2 compara las mismas variantes sobre esa instancia. El tiempo externo incluye el arranque del programa y la escritura de la salida. Un tiempo externo de 0.00 refleja el redondeo de `time`.

Variante	Ejecución	CPU (s)	Pared interna (s)	Pared externa (s)	Nodos
Básica	Interpretada	0.081	0.081	0.16	1458
Básica	Compilada	0.076	0.076	0.07	1458
Anticipada	Compilada	0.002	0.002	0.00	6

Tabla 2. Comparación de ejecución interpretada y compilada

6.4. Tamaños crecientes

Se tomaron los primeros grupos de `instances/benchmark/large.dat` hasta llegar a 10, 20, 40 y 80 sesiones. Cuando hizo falta, se redujo la cantidad semanal del último grupo. Se conservaron aulas, días, franjas y profesores, y se filtraron docencia y restricciones duras por los grupos presentes. Se quitaron `prefiere` y `compacta` para medir solo la búsqueda de un horario válido.

La tabla 3 muestra CPU en segundos y nodos como T/N. L indica el límite de 5000 nodos. El corte externo significa que no se obtuvo un reporte final en cinco segundos.

Sesiones	BT básico	BT original	BT MRV	FD MRV
10	0.256 / 5000 (L)	0.007 / 11	0.031 / 21	0.011 / 11
20	0.305 / 5000 (L)	0.022 / 21	0.218 / 21	0.060 / 21
40	0.353 / 5000 (L)	0.078 / 41	1.444 / 41	0.400 / 41
80	1.163 / 5000 (L)	0.473 / 81	Corte externo	2.446 / 81

Tabla 3. Medianas de CPU en segundos y nodos por tamaño

6.5. Optimización y pruebas de funcionamiento

En `instances/valid/optimization.dat` se deben elegir dos inicios distintos entre 1, 3 y 5. Sus costos son 6 para $(1, 3)$, 6 para $(1, 5)$ y 5 para $(3, 5)$. Ambas estrategias completaron la búsqueda y encontraron el mínimo de 5. La tabla 4 muestra sus mediciones.

Estrategia	Nodos	Retrocesos	CPU (s)	Pared (s)	Costo
BT	12	11	0.000	0.000	5
FD	7	6	0.000	0.001	5

Tabla 4. Optimización con *branch and bound*

Los ceros de CPU se deben a la resolución de un milisegundo. En el caso FD sin restricciones blandas, `instances/valid/small.dat`, la propagación asignó las dos sesiones y solo se contó el nodo inicial.

También se ejecutaron las nueve instancias originales con ambas estrategias. Solo se optimizaron `constraints.dat` y `optimization.dat`. Los resultados coinciden entre BT y FD.

Archivo relativo a `instances`	Resultado	Código	Sesiones	Costo
`valid/small.dat`	Válida	0	2	0
`valid/constraints.dat`	Óptima	0	5	0
`valid/optimization.dat`	Óptima	0	2	5
`benchmark/large.dat`	Primera válida	0	80	1
`invalid/syntax.dat`	Instancia inválida	1	-	-
`invalid/coherence.dat`	Instancia inválida	1	-	-
`unsatisfiable/load.dat`	Inconsistencia previa	1	-	-
`unsatisfiable/rooms.dat`	Sin solución	2	-	-
`unsatisfiable/restrictions.dat`	Sin solución	2	-	-

Tabla 5. Resultados de las instancias originales

6.6. Análisis de los resultados

En la instancia común, BT anticipado con orden original pasa de 1458 a 6 nodos y de 0.081 a 0.002 segundos de CPU. Esto equivale a una aceleración de 40.5 y a evitar el 99.59 % de los nodos. MRV, grado y demanda visitan los mismos nodos, pero tardan más en elegir la sesión. LCV explora 14 nodos y entrega una primera solución de costo 3. Su criterio busca dejar opciones disponibles, no reducir el costo blando.

Prolog prueba las cláusulas y alternativas en su orden. Ese orden influye en las opciones que se intentan primero. En la serie creciente, el orden original aprovecha la disponibilidad restringida y las sesiones agrupadas. MRV vuelve a revisar muchas sesiones pendientes. Con 40 sesiones visita los mismos 41 nodos, pero tarda 1.444 segundos frente a 0.078 del orden original. Con 80 sesiones alcanza el corte externo.

FD reduce los dominios, pero también prepara y prueba combinaciones y propaga restricciones entre variables. En esta serie, ese trabajo no siempre compensa. Con 80 sesiones, FD MRV y BT original visitan 81 nodos, pero tardan 2.446 y 0.473 segundos respectivamente.

Recorrer y copiar listas, probar opciones temporales y actualizar contadores con `assertz/1` y `retract/1` también toma tiempo. Esos pasos ayudan a explicar por qué visitar menos nodos no siempre produce la aceleración esperada. No se midió el costo de cada paso por separado.

La lectura y validación forman parte del tiempo interno. El código compilado evita cargar y compilar el programa en cada ejecución. El tiempo externo de la variante básica baja de 0.16 a 0.07 segundos, mientras la CPU interna cambia poco. La estructura de las instancias influye en los resultados, por lo que estos datos no demuestran que una heurística sea siempre mejor. Las diferencias de uno o dos milisegundos son pequeñas para sacar conclusiones con pocas repeticiones.

7. Limitaciones

- La variante básica se activa cambiando un hecho del código. No tiene una opción de comandos.
- La cota solo usa grupos completos y puede descartar pocas ramas.
- El diagnóstico no busca un conjunto mínimo ni tiene límite de nodos. Tampoco se limitan el tiempo y la memoria de preparación, heurísticas y diagnóstico.
- No se detectan enteros demasiado grandes. El verificador no exige inicios enteros en horarios recibidos directamente. Los buscadores sí generan enteros.
- Las rutas de entrada y salida pueden coincidir y sobrescribir archivos. Un fallo al escribir puede dejar archivos incompletos o solo el reporte, sin la exportación.